AI-ASSISTED PROCESS DOCUMENTATION

Quarters of process documentation in hours of SME time — documented and traceable as you go.

Processminer pairs your subject-matter expert with an AI interviewer, drafts every section as you go, and shows the **source for every sentence**. The SME approves block by block. The documentation is complete and traceable the moment the session ends — not weeks of write-up later.

THE PROBLEM

Process docs take a quarter — and you've got nothing usable until the very end.

Senior SME time is the bottleneck. "Just let AI write it" sounds great until the AI confidently invents the part nobody actually said. By the time you spot it, the wrong draft is already in three downstream decks.

THE PITCH

The AI does the typing. The SME does the saying.

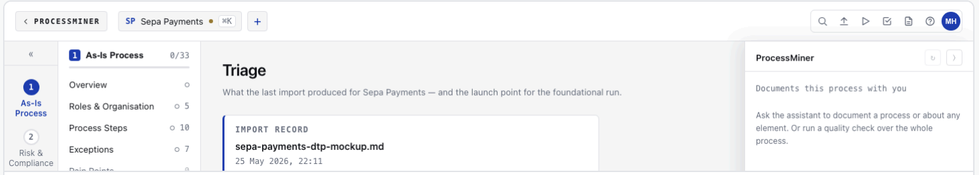
Every sentence shows who said it and where. What's confirmed looks confirmed. What's still a draft looks like a draft. Nothing is hidden, nothing is invented, nothing ships without your SME's nod.

said by SME · from document · AI draft

THE OUTCOME

One source of truth covering six perspectives — ready the moment the SME closes the session.

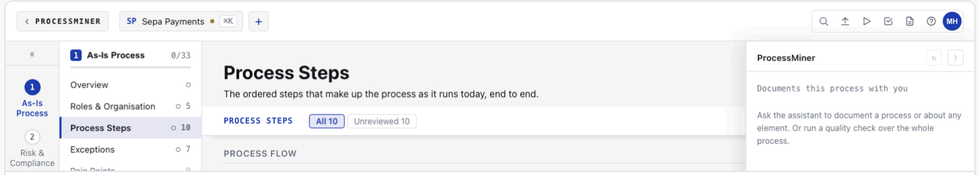
The same record serves the transformation lead, the operations team, and whoever picks up the process next. No parallel decks, no stale Confluence pages, no "let me find the latest version."



STEP 1 · DROP THE DOCUMENT IN

The AI reads it and proposes a first draft.

You see exactly what's covered, what's still open, and what's worth a closer look — before you spend any SME time.



STEP 2 · WALK IT WITH THE SME

The AI proposes. The SME confirms, corrects, or cuts.

Every step shows its source. "AI-drafted · awaiting confirmation" is honest, not hidden. The SME stays in charge.

What you get

- Standardised documentation
- Live capturing with the SME
- No endless workshops anymore

Hours of SME time, not quarters of write-up | Documented inline, not weeks later | Every claim cited at the heading where it lives

HOW THE TOOL IS ORGANIZED

ProcessMiner documents what is. ArchitectMiner designs what should be.

One platform, two specialist modules. They share the same wiki, the same SME, and the same audit trail — but they answer different questions and serve different roles. A documented process flows from the first module straight into the second; nothing is re-keyed.

MODULE 1 · THE PROCESS

ProcessMiner

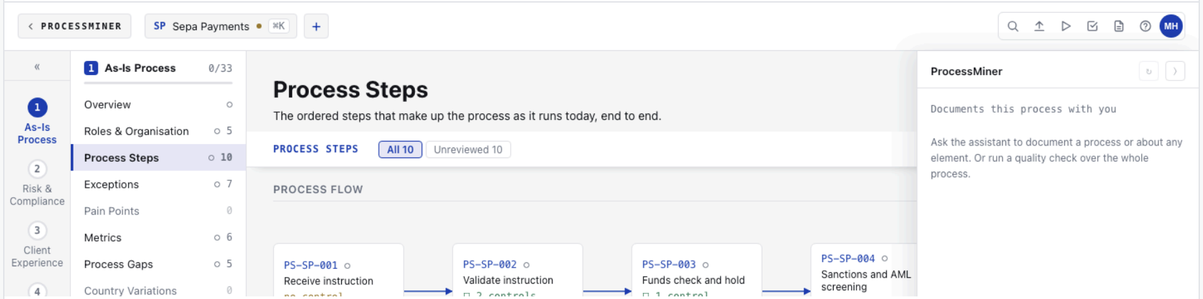
The business view — how the process runs today, and what it should look like.

Pairs the SME with an AI interviewer to **document the process end-to-end across six perspectives**, then develops the target state with the same SME — controls, regulations, customer experience, innovation, transformation.

SIX SECTIONS IT OWNS

- 01 Process & operations
- 02 Risk & compliance
- 03 Client experience
- 04 Innovation
- 05 Target process
- 06 IT landscape (As-Is)

Run by · process specialists, compliance officers, client-journey designers, innovation analysts, transformation leads.



MODULE 2 · THE ARCHITECTURE

ArchitectMiner

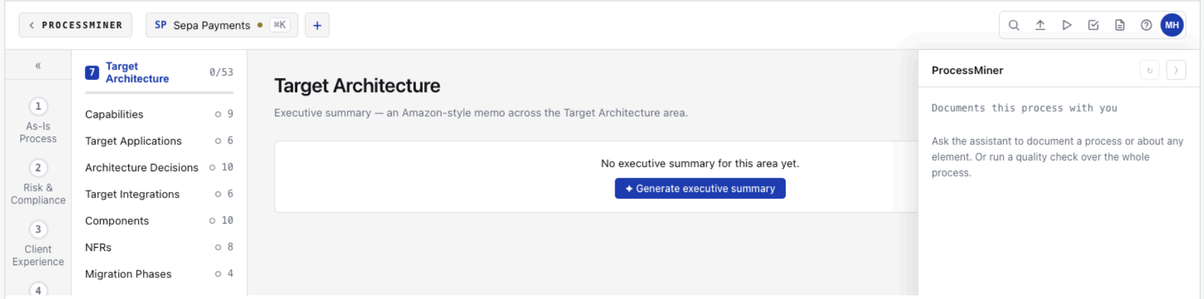
The IT view — the target architecture that will support the target process.

Takes the documented process and the target state, then **develops a seven-section target architecture** — capabilities, target applications, ADRs, integrations, components, NFRs and the migration phases to get there.

SEVEN SECTIONS IT OWNS

- 01 Capabilities
- 02 Target applications
- 03 Architecture decisions
- 04 Target integrations
- 05 Components
- 06 Non-functional requirements
- 07 Migration phases

Run by · domain architects, solution architects, enterprise architects.



INPUT
Document + SME

→

PROCESSMINER
As-Is & target process

→

ARCHITECTMINER
Target architecture

→

OUTPUT
Audit · transformation · board

Shared wiki — both modules read and write the same per-process record | Same provenance rules apply to both | Switch modules with the toggle at the top of the dashboard

KARPATHY'S LLM WIKI — WHAT HE PROPOSED

Treat knowledge as a structured artifact, not a retrieval target.

In a May 2026 gist, **Andrej Karpathy** (ex-Tesla AI director, OpenAI co-founder) sketched an alternative to RAG for knowledge work: let the wiki *be* the artifact, keep it as typed Markdown on disk, and let the LLM read and write it directly under schema constraints. karpathy/442a6bf… · Processminer applies that pattern to process documentation, then adds per-heading provenance and an approval gate on top.

WHY HE WROTE IT

Naïve RAG hides what the LLM made up.

LLMs are confidently fluent — their output looks finished even when it's wrong. The standard recipe (`chunk → embed → retrieve top-k → stuff into context`) lets the model rephrase retrieved fragments into plausible prose with no clear line between source-true and model-invented.

The result reads beautifully. But you can't tell which sentence came from a source and which came from the model's prior. **In regulated knowledge work, that's disqualifying.**

THE CORE MOVE

Make the wiki the artifact. The schema is the contract.

Knowledge-work products already *are* wikis: typed sections, named files, structured fields. Stop pretending knowledge is just chunks waiting to be retrieved. **Let the wiki be the artifact.**

The LLM reads it directly with file-level tools. The schema defines what's writable. The LLM emits structured specs; deterministic code writes the files. Reading and writing follow completely different rules.

```
wiki = artifact · schema = contract · LLM = drafting partner
```

THE READ/WRITE RULE

Judgement is the model's. Mechanics are deterministic code.

READ — the LLM reads the wiki directly with `Read` / `Grep` tools. No vector store, no embeddings, no retrieval ranking. The wiki is small (10^2 – 10^3 elements) and structured; grep on disk beats nearest-neighbour search.

WRITE — the LLM never touches Markdown. It emits a JSON spec; a schema-validated Python writer accepts or rejects it. Two runs of the same skill on the same input produce `byte-identical` files.

Three file layers. Each owns one thing.

Karpathy's pattern, instantiated for Processminer. The model has judgement; the files have truth; the schema has the contract.

L1	**Raw sources —** immutable The original uploaded documents. The receipts. Never modified by the system.	`raw-sources/sepa-payments/dtp-2025-q4.md` `raw-sources/sepa-payments/sme-transcript.txt`
L2	**The wiki —** typed Markdown + JSON sidecars The shared workspace. One folder per process, one file per element. Every heading carries provenance.	`wiki/processes/sepa-payments/process-steps/P…` `wiki/processes/sepa-payments/provenance.json`
L3	**The schema —** single source of truth 32 element types, 11 relation types, the provenance contract. The only place that defines what's valid.	`schema/process-schema.json · 2,200 lines` `schema/.derived/process-step.llm.json`

PROCESSMINER'S EXTENSION TO THE PATTERN

Per-heading provenance + an approval gate makes it auditable.

Karpathy's pattern says "track provenance." Processminer makes provenance a first-class schema field that the approval gate consults.

- **Provenance is per heading, not per element.**
  Every named section in every element carries `{source, verbatim quote}` — six source types: `elicited`, `document`, `web`, `proposed`, `legacy-approved`, `imported`.
- **Approval is gated on every heading.**
  Nothing reaches `approved` while any heading is `proposed`. The wiki tells you what still needs SME confirmation, at heading granularity.
- **Edits auto-flip the heading back to draft.**
  Touch a confirmed heading and its provenance flips to `proposed`. No silent re-breaking of an approved record.

> *"The wiki is the artifact, not the retrieval system."*
>
> — A. Karpathy, LLM Wiki gist · paraphrased

WHAT RUNS AUTONOMOUSLY — AND WHERE THE DATA GOES

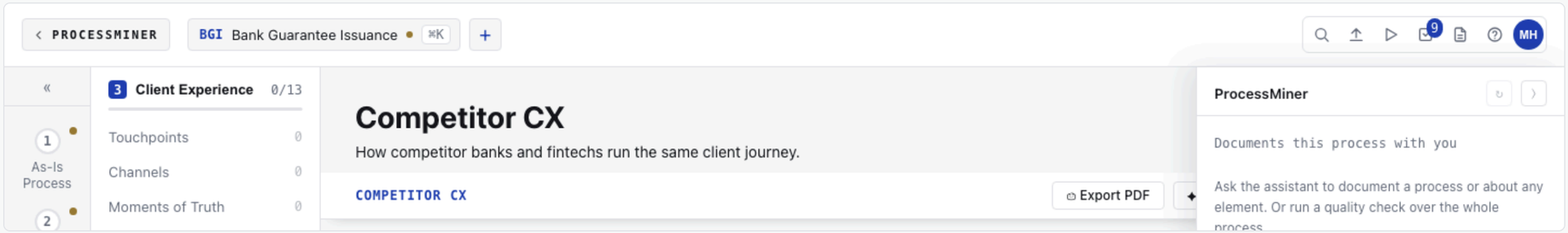
Auto-research pre-fills the obvious. The SME approves the verdict. The wiki stays local.

Four autonomous web-research skills scrape the open web for what's already known about the process — regulation, competitor moves, CX benchmarks. The SME never wastes time on desk research. Every web finding is gated behind the same approval rules; the process knowledge never leaves the analyst's filesystem.

SAMPLE OUTPUT · /SOURCE-CX

Real competitor cards, auto-researched.

Each card names the competitor, stamps the source URL, and lives in the wiki as `source: web` until the SME confirms it. **Auto-research is the runway, never the destination.**



AUTO-RESEARCH · THE WEB IS A SOURCE, NOT THE SOURCE

Four autonomous skills pre-fill what the open web already knows.

The AI does the desk research **before** the SME walks in — so the SME's time goes into validating, correcting, and adding the inside knowledge only they have.

`/source-regulation`	Financial-services **regulation, supervisory rules and guidance** that govern the process. Stamped with the regulator and the source URL.
`/source-cx`	Competitor banks & fintechs running the same client journey + **industry CX benchmarks**. Each finding cites the competitor.
`/source-innovation`	Market trends, analyst reports, **competitor moves** in the space. Useful for the innovation perspective and target-state planning.
`/source-target`	Drafts an initial **target process** from the documented As-Is + risk + CX + innovation perspectives — gives the SME something concrete to react to.

Approval is still gated. Every web finding lands as `source: web` with the URL stamped on the heading. Nothing reaches `approved` until the SME confirms it — same rule as any other source. **Auto-research is the runway, never the destination.**

TRUST MODEL · WHERE THE DATA LIVES (AND DOESN'T)

The bank's process knowledge stays on the bank's filesystem.

Standard LLM tradeoff, made explicit: **files local, inference remote**. The architecture supports swapping to an on-prem model when the bank asks for air-gapped operation.

- **Wiki + source documents** live as files on the analyst's machine.
  No database, no vector store, no cloud sync. `git` tracks every change — that *is* the audit log.
- **Model inference** runs via an external LLM provider.
  A local AI-agent runtime keeps long-lived sessions warm (6 by default). Per-turn context is sent on each turn; verify your provider's retention policy.
- **No RAG, no embeddings.**
  The AI reads the wiki directly with `Read` / `Grep` tools. The wiki is small enough — grep on disk beats vector search at this scale.
- **Auth + access control on top.**
  Per-user sessions, role-based process ownership, password-gated login. Per-process access lists ship as `data/process-access.json`.